

Technology Stack

Date	02/07/2026
Team ID	SWTID-2026-5714
Project Name	OptiCrop: Smart Agriculture Production Optimization Engine
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the solution. A well-chosen technology stack ensures that the application is scalable, maintainable, and aligned with the project requirements.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / ClientSide	HTML5, CSS3 (customstyled, no framework)	A lightweight, dependency-free frontend keeps the app simple to deploy and maintain. Custom CSS (dark soilgreen theme with Fraunces/Space Grotesk typography) gives a distinctive look without the overhead of a JS framework, since the app is a straightforward multi-page form-andresult flow with no complex client-side state.
2	Backend / ServerSide	Python + Flask, Scikit-learn	Python is used for machine learning and backend development, while Flask provides a lightweight web framework to integrate the trained Random Forest model with the user interface. This combination keeps the application simple, fast, and easy to maintain.
3	Database / Data Storage	Static CSV file (Crop_recommendation.csv) + Pickle (.pkl) files	The dataset (2,200 rows) is small and unchanging, so a CSV file is sufficient — no database server is needed for this scale. The trained model and scaler are serialized with Pickle and loaded directly into memory at app startup, avoiding the overhead of a database or external storage service.

4	Cloud / Hosting / Deployment	Render (free tier)	Render offers simple, direct deployment from a GitHub repository with no infrastructure to manage — well suited to a project of this scale. It automatically installs dependencies from requirements.txt and runs the app via gunicorn. The free tier does spin down after inactivity,
---	-------------------------------------	--------------------	--

Below is a detail of the specific technologies chosen for each component of the architecture, with a brief justification for each choice.

Technology Stack Details

			causing a 30-50 second cold-start delay, which is an accepted tradeoff for a no-cost hosting solution.
5	Version Control & CI/CD	Git + GitHub	Git and GitHub provide version history and a public repository link for the project submission. No CI/CD pipeline (e.g. GitHub Actions) is currently set up, since deployments are manual pushes to Render — appropriate for a single-developer project at this scale, though automated testing/deployment would be a reasonable next step if the project grew.
6	Third-Party APIs / Other Tools	None currently integrated	The current version does not call any external APIs (no live weather, SMS, or maps services) — all predictions are based solely on the user's manually entered form values and the trained model. Integrating a live weather API (e.g. to auto-fill temperature/humidity/rainfall) is listed as a future enhancement, not part of the current build.

